```
"""
compareModel() - Multi-Model Comparison & Selection Pipeline
-----------------------------------------------------------------
Trains and evaluates four classification models (Decision Tree, Random
Forest, KNN, and XGBoost), compares their performance side by side, and
selects the best model for deployment based on accuracy and robustness.

The chosen model is saved to disk using pickle for later use in a Flask
application or any other deployment environment.
"""

import pickle
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score, confusion_matrix, classification_repo

try:
    from xgboost import XGBClassifier
    XGBOOST_AVAILABLE = True
except ImportError:
    XGBOOST_AVAILABLE = False
    print("[WARNING] xgboost is not installed. Run 'pip install xgboost' to enabl


def compareModel(X_train, X_test, y_train, y_test, random_state=42):
    """
    Trains Decision Tree, Random Forest, KNN, and XGBoost models,
    evaluates each on the test set, and prints a consolidated
    comparison of their performance.

    Parameters:
        X_train, X_test : Training/testing feature sets
        y_train, y_test : Training/testing target labels
        random_state    : Seed for reproducibility (default=42)

    Returns:
        results     : dict mapping model name -> {model, y_pred, accuracy}
        best_model_name : name of the model with the highest accuracy
    """
    models = {
```

```
        "Decision Tree": DecisionTreeClassifier(random_state=random_state),
        "Random Forest": RandomForestClassifier(n_estimators=100, random_state=ra
        "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
    }

    if XGBOOST_AVAILABLE:
        models["XGBoost"] = XGBClassifier(
            use_label_encoder=False, eval_metric="logloss", random_state=random_s
        )

    results = {}

    print("\n========== MODEL COMPARISON ==========")
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        cm = confusion_matrix(y_test, y_pred)
        cr = classification_report(y_test, y_pred)

        results[name] = {"model": model, "y_pred": y_pred, "accuracy": acc}

        print(f"\n--- {name} ---")
        print(f"[RESULT] Accuracy: {acc:.4f}")
        print("Confusion Matrix:")
        print(cm)
        print("Classification Report:")
        print(cr)

    # Select the best model based on accuracy (ties broken by preferred order)
    preferred_order = ["XGBoost", "Random Forest", "Decision Tree", "K-Nearest Ne
    best_acc = max(r["accuracy"] for r in results.values())
    tied_models = [name for name, r in results.items() if r["accuracy"] == best_a
    best_model_name = next((name for name in preferred_order if name in tied_mode

    print("\n========== SUMMARY ==========")
    for name, r in sorted(results.items(), key=lambda x: -x[1]["accuracy"]):
        print(f"{name:<22} Accuracy: {r['accuracy']:.4f}")

    print(f"\n[SELECTED] Best model for deployment: {best_model_name}")

    return results, best_model_name


def save_model(model, filepath="best_model.pkl"):
    """Saves a trained model to disk using pickle."""
```

```
    with open(filepath, "wb") as f:
        pickle.dump(model, f)
    print(f"[INFO] Model saved to {filepath}")


# ---------------------- Example Usage ----------------------
if __name__ == "__main__":
    # Generate a synthetic dataset (replace this with your own data)
    X, y = make_classification(
        n_samples=500,
        n_features=10,
        n_informative=6,
        n_redundant=2,
        random_state=42
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    results, best_model_name = compareModel(X_train, X_test, y_train, y_test)

    # Save the best model for deployment (e.g. in a Flask app)
    best_model = results[best_model_name]["model"]
    save_model(best_model, "best_model.pkl")
```